



# Agent Self-Evolution

Phase 2 Validation Report — Closed-loop-aware deploy gate + tool-side parity

---

Date: May 25, 2026

Repository: [github.com/jramos/agent-self-evolution](https://github.com/jramos/agent-self-evolution)

# Executive Summary

Agent Self-Evolution is a standalone optimization pipeline that uses DSPy and GEPA (Genetic-Pareto Prompt Evolution) to automatically improve an agent's skills, tool descriptions, system prompts, and code through evolutionary search — all via API calls with no GPU training required. Phase 1 shipped the framework's first deploy gate (synthetic-only, paired-bootstrap CI, knee-point selection). Phase 2 makes the gate *behavior-aware* — it can now ship candidates whose synthetic signal is flat or slightly negative when the closed-loop behavioral signal demonstrably improves — and ships *tool-side parity*, so the same gate, audit trail, and automation run against tool descriptions as well as skill files.

This report documents the Phase 2 validation of the closed-loop-aware deploy gate. We evolved the **weakened-systematic-debugging** skill end-to-end with the gpt-5.4-mini optimizer stack and validated the candidate against a five-task behavioral suite executed end-to-end by the openai/gpt-5-mini validator agent. The synthetic holdout delta came in tiny-and-negative ( $0.976 \rightarrow 0.972$ ,  $\Delta -0.004$ ) — under Phase 1's strict-synthetic gate this candidate would have been rejected. The closed-loop signal told a different story: behavioral pass-rate went from **2/5** to **5/5** (gained **+3** tasks, required  $\geq 1$ ). The new gate consulted that signal directly and decided **DEPLOYED** via the closed-loop behavioral signal; the synthetic sanity check passed (delta within the  $\pm 0.05$  tolerance, so the synthetic regression is confirmed within noise).

KEY RESULT — weakened-systematic-debugging (skill-side deploy via CL-aware gate)

Synthetic holdout (n=20): 0.976 → 0.972 ( $\Delta -0.004$ )

Closed-loop tasks: 2/5 → 5/5 (+3, required  $\geq 1$ )

Synthetic sanity check: passed ( $\Delta$  within  $\pm 0.05$ )

Decision: **DEPLOYED** via the closed-loop behavioral signal

## Background

Agent Self-Evolution targets the instructions layer of an LLM agent — skill files, tool descriptions, and system prompts — and evolves the text via API-only evolutionary search. The framework was originally built for Hermes Agent (Nous Research) but a pluggable SkillSource protocol now discovers artifacts in the Hermes Agent layout, the Claude Code plugin cache, or any flat local directory. An agent's behavior is governed by three layers:

| Layer | What It Is | How It's Currently Improved |
|-------|------------|-----------------------------|
|-------|------------|-----------------------------|

|               |                                           |                              |
|---------------|-------------------------------------------|------------------------------|
| Model Weights | The underlying LLM (Claude, GPT, etc.)    | RL training (Tinker-Atropos) |
| Instructions  | Skills, system prompts, tool descriptions | Manual authoring (static)    |
| Tool Code     | Python implementations of each tool       | Manual development           |

Phase 1 validated the framework end-to-end on the instructions layer with a synthetic-only deploy gate. Phase 2 extends that gate in two ways. First, the gate is now **behavior-aware**: alongside the synthetic LLM-as-judge holdout, every candidate is exercised on a closed-loop behavioral suite — real tasks scored by a validator agent — and the gate can deploy on the behavioral signal directly when the synthetic signal is flat. Synthetic eval can saturate (every candidate scores near 1.0) or drift; the closed-loop signal answers the question that actually matters at deploy time: *does this candidate help the agent succeed at real tasks?* Second, the same gate, runner, and automation now apply to **tool descriptions** via `evolve_tool.py`, achieving full parity with the skill-side `evolve_skill.py` pipeline.

## Approach: Evolutionary Skill Optimization

### Three Optimization Engines

| Engine            | What It Optimizes                   | License | Role                     |
|-------------------|-------------------------------------|---------|--------------------------|
| DSPy + GEPA       | Skills, prompts, tool descriptions  | MIT     | Primary (validated)      |
| DSPy MIPROv2      | Few-shot examples, instruction text | MIT     | Fallback optimizer       |
| Darwinian Evolver | Code files, algorithms              | AGPL v3 | Code evolution (Phase 4) |

**GEPA** (Genetic-Pareto Prompt Evolution) is the star engine — an ICLR 2026 Oral paper from Stanford/UC Berkeley. Unlike traditional evolutionary search that only sees pass/fail scores, GEPA reads full execution traces to understand *why* things failed, then proposes targeted mutations. It outperforms reinforcement learning (GRPO) by +6% with 35x fewer rollouts, and outperforms DSPy's previous best optimizer (MIPROv2) by +10%. It works with as few as 3 training examples. Phase 2 ships two refinements to how GEPA is driven: a **saturation pre-flight** that refuses to spend budget on baselines with no headroom, and an **improvement-or-equal acceptance criterion** that halves the false-rejection rate at the noise floor.

### The Optimization Pipeline

- Saturation pre-flight** — Score the baseline on a sample of synthetic + closed-loop examples; refuse to run if the baseline lands in the `no_headroom`, `uniform_failure`, or `weak_signal` band (cost-saving guard added this cycle)

2. **Discover and load artifact** — Resolve the skill (or tool) via the SkillSource / ToolSource protocol, parse YAML frontmatter and body
3. **Generate eval dataset** — An LLM reads the artifact and synthesizes (task, expected\_behavior) pairs, then splits into train / val / holdout
4. **Wrap as DSPy module** — The artifact text becomes a parameterized DSPy module where the instructions are the optimizable parameter
5. **Run optimizer** — DSPy GEPA evolves the instructions with improvement-or-equal acceptance, scored by an LLM-as-judge with a structured rubric
6. **Knee-point selection** — Among candidates within  $\epsilon$  of the val-best, pick the highest-val candidate (smallest body as tiebreak)
7. **Dual-signal deploy gate** — Score the candidate on the synthetic holdout (paired-bootstrap CI) AND execute it on a closed-loop behavioral suite; the gate consults a decision\_signal field and may deploy via the closed-loop path when synthetic is flat but behavior gains  $\geq$  required threshold
8. **Report** — Structured gate\_decision.json (v5 schema, with CL-primary fields), before/after artifacts, full LM trace log, opt-in --create-pr automation

The **saturation pre-flight** is the new cost-control story for Phase 2. Synthetic LLM-as-judge fitness saturates aggressively at this validator tier: in preparing this report we ran the pre-flight against three candidate headline artifacts and two of them — search\_files (a tool description) and a deliberately-weakened write\_file suite — were correctly refused as saturated against the validator's gpt-5.4-mini tier. The pre-flight kept us from spending GEPA budget on baselines GEPA cannot beat. The headline run reported here (**weakened-systematic-debugging**) cleared the pre-flight in the weak\_signal band — the band the gate was redesigned for — and consumed **\$1.27** across 689 LM calls in ~88 minutes. **Tool-side parity** is the second Phase 2 deliverable: evolve\_tool.py ships the same GEPA runner, dataset builder, quality-gate, audit trail, and opt-in PR automation as evolve\_skill.py. The headline result lands skill-side because the closed-loop suites we have for tool-side surfaces are saturated for our current validator tier — the deliverable is full parity, ready for harder tool-side eval surfaces.

## Phase 2 Experiment

### Configuration

| Parameter     | Value                                                                                                                                                                             |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Target Skill  | weakened-systematic-debugging (weakened systematic-debugging — deliberately-weakened baseline that lands in the weak_signal saturation band, exercising the CL-aware deploy path) |
| Baseline Size | 1,987 characters                                                                                                                                                                  |

|                         |                                                                                                          |
|-------------------------|----------------------------------------------------------------------------------------------------------|
| Optimizer LM            | openai/gpt-5.4-mini                                                                                      |
| Reflection LM (GEPA)    | openai/gpt-5.4-mini                                                                                      |
| Eval / Judge LM         | openai/gpt-5.4-mini                                                                                      |
| Optimizer               | DSPy GEPA (light budget; improvement-or-equal acceptance)                                                |
| Synthetic Eval Set      | 53 examples (18 train / 15 val / 20 holdout)                                                             |
| Total Optimization Time | 5,305 seconds (~88 minutes)                                                                              |
| Total LM Calls          | 689 (from metrics.json per-model summary)                                                                |
| Total Cost (USD)        | \$1.27                                                                                                   |
| Quality Gate            | dual-signal — synthetic holdout (paired-bootstrap CI) + closed-loop behavioral suite (CL-primary on tie) |
| Knee-point Strategy     | val-best (default; smallest available via --knee-point-strategy)                                         |
| Closed-loop Validator   | openai/gpt-5-mini                                                                                        |
| Closed-loop Suite       | 5 tasks (behavioral benchmark, scored end-to-end)                                                        |

## Evaluation Dataset

The evaluation dataset was synthetically generated by openai/gpt-5.4-mini. Given the full weakened-systematic-debugging SKILL.md text, the model produced 53 realistic test cases with rubric-based expected behaviors, then split them into train / val / holdout per the framework's configured ratios. The closed-loop validation suite is a separate five-task `systematic_debugging.jsonl` behavioral benchmark executed end-to-end by the validator agent — those tasks are not part of the synthetic split and are evaluated only on the baseline and the final knee-point candidate. Examples drawn from the synthetic train split:

| Task Input                                                                                                                                                                                    | Expected Behavior (Rubric)                                                                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The function <code>`average_nonzero(nums)`</code> in <code>`analysis.py`</code> divides by <code>`len(nums)`</code> even though it should ignore zeros. Diagnose the bug and specify the fix. | State that the denominator is wrong because zeros are included, explain current vs intended behavior, and propose filtering zeros before computing the mean.         |
| The function <code>`median(values)`</code> in <code>`stats.py`</code> sorts the input list in place. A test expects the original list to remain unchanged. Diagnose the bug.                  | Explain that in-place sort mutates caller data, identify the sort line, state intended behavior should preserve input sequence, and propose sorting a copy instead.  |
| In <code>`prime.py`</code> , <code>`is_prime(n)`</code> returns True for 1. Diagnose the bug and specify the exact fix.                                                                       | Identify missing guard for $n < 2$ , explain current behavior incorrectly treats 1 as prime, intended behavior excludes 0 and 1, and propose adding an early return. |

## Fitness Function

Synthetic fitness is measured by an LLM-as-judge (gpt-5.4-mini) that scores each candidate output along three rubric dimensions. The composite score is a weighted combination with a

length-penalty term that discourages runaway expansion:

$$\text{composite} = 0.5 \cdot \text{correctness} + 0.3 \cdot \text{procedure\_following} + 0.2 \cdot \text{conciseness} - \text{length\_penalty}$$

The judge also returns a free-text feedback string that GEPA's reflection LM consumes to propose targeted instruction-text mutations on the next iteration — this trace-aware loop is the core of GEPA's sample efficiency. Phase 2 adds a second, independent fitness signal at gate time: **closed-loop behavioral pass-rate** on a small held-out task suite executed by a validator agent. The gate consults both signals — synthetic for sanity ( $\pm 0.05$  tolerance), closed-loop for the deploy decision when synthetic is flat.

## Results

| Metric                   | Baseline | Evolved (knee-point pick) | $\Delta$           |
|--------------------------|----------|---------------------------|--------------------|
| Body size (chars)        | 1,987    | 2,967                     | +49.3%             |
| Avg holdout score (n=20) | 0.976    | 0.972                     | -0.004             |
| Bootstrap mean diff      | —        | -0.004                    | —                  |
| Bootstrap 90% CI lower   | —        | -0.028                    | —                  |
| Bootstrap 90% CI upper   | —        | +0.012                    | —                  |
| Closed-loop tasks (n=5)  | 2/5      | 5/5                       | +3 (req $\geq 1$ ) |
| Decision                 | —        | DEPLOYED                  | via closed-loop    |

The evolved weakened-systematic-debugging skill grew **+49.3%** (1,987  $\rightarrow$  2,967 chars) and the synthetic holdout score moved **-0.004** (0.976  $\rightarrow$  0.972) on n=20 examples. The synthetic delta is tiny-and-negative — under Phase 1's no-regression rule, this candidate would have been rejected. Phase 2's behavior-aware gate looked at the closed-loop signal instead: behavioral pass-rate went from **2/5** tasks to **5/5** (gained **+3**, required  $\geq 1$ ). The synthetic sanity check passed (delta inside the  $\pm 0.05$  noise envelope), so the synthetic regression is statistically indistinguishable from noise while the behavioral improvement is large and concrete. **Decision: DEPLOYED** via the closed-loop behavioral signal. This is the textbook case the Phase 2 gate was redesigned for.

## How the Result Was Produced

GEPA evolves skill instructions through a reflective loop; Phase 2's gate then reads two independent signals at decision time:

1. Run candidate skill instruction text on training examples; the judge scores each output and emits free-text feedback
2. Reflection LM reads the execution traces + feedback and proposes a targeted mutation of the instruction text (Phase 2: improvement-or-equal acceptance keeps near-ties in the

population)

3. Score the mutated candidate on the validation set (15 examples); track every candidate's per-example Pareto front
4. After GEPA's light-budget search, freeze the candidate population; knee-point selection picks candidate 1 (val=0.987, rank 1 of 2 in the  $\epsilon$ -band, 2,753 body chars) and is also the candidate GEPA's own default selector would have chosen — the val-best and GEPA-default converged here
5. **Dual-signal gate** — Score the knee-point pick on the 20-example synthetic holdout (paired-bootstrap CI on per-example diffs) AND execute it on the closed-loop behavioral suite (5 tasks, scored by the openai/gpt-5-mini validator). The gate sets `decision_signal` based on which signal carries the decision; on this run synthetic was flat-within-tolerance and closed-loop gained 3 tasks, so the gate deployed via the closed-loop path.

Three Phase 2 design choices made this outcome possible: (a) the dual-signal gate deploys on either signal, so a flat synthetic doesn't veto a real behavioral gain; (b) the synthetic sanity check still guards against unambiguous synthetic regressions ( $\pm 0.05$  tolerance); (c) the saturation pre-flight refused two of three candidate headline artifacts before this run, which is honest evidence that the current validator tier saturates aggressively on our existing eval surfaces — the Phase 2 gate exists specifically to recover deploy decisions on the artifacts that do clear the pre-flight in the *weak\_signal* band. The CL-aware deploy gate arc is supported by a broader May-cycle calibration campaign across nano-pdf, apple-notes, polymarket, and huggingface-hub (reports/calibration\_findings.md) — that campaign contributed the improvement-or-equal acceptance default, retired the knee-point  $\epsilon$  selector as a no-op on val-best, and recommended the non-inferiority tolerance sweet spot used by the synthetic sanity check.

## Safety and Guardrails

Every evolved variant must pass all of the following constraints before deployment:

| Constraint                | Enforcement                                                     | Status      |
|---------------------------|-----------------------------------------------------------------|-------------|
| Self-evolution test suite | 1,166 pytest tests pass on the optimizer itself                 | Implemented |
| Static size limits        | Skills $\leq 15$ KB, tool descs $\leq 500$ chars (configurable) | Implemented |
| Absolute char ceiling     | Hard cap on evolved artifact size (default 5,000)               | Implemented |
| Growth-quality gate       | Required improvement scales linearly with growth %              | Implemented |
| Paired-bootstrap CI       | 90% CI on per-example holdout diffs gates deploy                | Implemented |

|                           |                                                                                                   |             |
|---------------------------|---------------------------------------------------------------------------------------------------|-------------|
| Knee-point selection      | Smallest candidate within $\epsilon$ of val-best                                                  | Implemented |
| Structural integrity      | Valid YAML frontmatter required                                                                   | Implemented |
| Deployment via PR         | Human review required, never auto-merge                                                           | By design   |
| CL-aware deploy gate      | Deploys via closed-loop signal when CL gain $\geq$ required AND synthetic delta within $\pm 0.05$ | Implemented |
| Saturation pre-flight     | Refuses to spend budget on no_headroom / uniform_failure / weak_signal baselines                  | Implemented |
| GEPA improvement-or-equal | Candidates accepted on $\geq$ (not strict $>$ ); halves false-rejection at the noise floor        | Implemented |
| PR automation audit trail | Opt-in --create-pr opens draft PR; pr_created field logs branch + SHA + URL                       | Implemented |
| Benchmark regression      | TBLite / skill-specific harness must hold                                                         | Planned     |

Source skill and tool repositories are never modified directly. All evolution output (evolved artifacts, gate decisions, run logs, closed-loop validation transcripts) is written under the framework's local output/ directory, and improvements are proposed as draft pull requests against the source repo for human review.

## Roadmap

| Phase   | Target                   | Engine             | Timeline  | Status      |
|---------|--------------------------|--------------------|-----------|-------------|
| Phase 1 | Skill files (SKILL.md)   | DSPy + GEPA        | 3-4 weeks | Validated ✓ |
| Phase 2 | Tool descriptions        | DSPy + GEPA        | 2-3 weeks | Validated ✓ |
| Phase 3 | System prompt sections   | DSPy + GEPA        | 2-3 weeks | Planned     |
| Phase 4 | Tool implementation code | Darwinian Evolver  | 3-4 weeks | Planned     |
| Phase 5 | Continuous improvement   | Automated pipeline | 2 weeks   | Planned     |

Each phase must demonstrate measurable improvement and pass benchmark regression gates before proceeding. Phase 2 ships the deploy gate's behavioral awareness AND tool-side parity — the gate is no longer hostage to synthetic-eval saturation, and the same automation that gated skill evolution now gates tool-description evolution. Phase 3 (system-prompt sections) is the next surface; Phase 5 (continuous improvement) closes the loop with automated cron-driven optimization.

## Immediate Next Steps

1. **Phase 3 scoping** — System-prompt sections as the next evolvable surface. The instructions layer remains the target; system prompts complete the trio of skills / tools / system prompts and are the highest-leverage instructions surface for most agents.
2. **Eval-surface hardening** — Develop harder closed-loop suites and synthetic generators so the saturation pre-flight is less often the dominant outcome at the gpt-5.4-mini validator tier. The Phase 2 gate works; the bottleneck is now the eval surfaces feeding it.
3. **Cross-tool portfolio campaign** — Run the CL-aware deploy gate against the realistic manifest's tool descriptions to surface which tool-side artifacts have headroom under the current validator tier — analogous to the May skill-side calibration campaign that produced the improvement-or-equal acceptance default.
4. **Phase 5 — continuous improvement** — Cron-driven optimization with budget gates, alerting, and an opt-in PR-automation queue. The `--create-pr` primitive shipped in Phase 2 is the prerequisite; Phase 5 wires it into a scheduled loop with backstop budgets.
5. **Documentation polish** — Link the Phase 2 report from the README roadmap section and the framework's headline narrative; refresh the architecture diagram to reflect the dual-signal gate.